Subject: Cloud Computing (CC)

Docker Hands-On Lab Assignment

A 2-Hour Hands-On Lab: Containers, Images, Dockerfiles & Registries

Neel Prajapati

Enrollment No: 202611022

Lab Reference: https://techifylabs-docker.learnwithdutta.com/

Contents

1 Introduction **2**

2 Setup Check **2**

3 Step 1: Run Your First Container **3**
3.1 Docker Installation Check in WSL2 3
3.2 Running First Nginx Container 3
3.3 Verifying with docker ps, docker logs 4

4 Step 2: Images vs Containers **4**
4.1 Pull, Create, Start Stages 4
4.2 One Image, Many Containers 5

5 Step 3: Look Inside a Container **5**
5.1 Exploring Container Internals 5

6 Step 4: Who Actually Does the Work? **5**
6.1 Docker Architecture 6
6.2 Stopping the Daemon 6

7 Step 5: Setting Limits (cgroups) **7**

8 Step 6: Layers and Copy-on-Write **8**

9 Step 7: Build Your Own Image **9**
9.1 Writing a Dockerfile for CampusConnect 9
9.2 Fixing the Dockerfile 10

10 Step 8: Keep Your Data (Volumes) **10**

11 Step 9: Ship It Like a Pro **11**

12 Conclusion **12**

Introduction

This report documents the Docker Hands-On Lab Assignment completed as part of the Cloud Computing (CC) course. The lab consists of nine carefully structured steps designed to provide practical, hands-on experience with Docker containers, images, Dockerfiles, volumes, and container registries.

The lab was conducted following the instructions provided at https://techifylabs-docker.learnwithdutta.com/. The objective was to understand containerization concepts through direct experimentation — running containers, examining their internals, building custom images, managing persistent data, and deploying production-ready containers.

Key concepts covered include:

- Containers vs Virtual Machines
- Docker architecture (CLI, daemon, containerd, shim, runc)
- Namespaces and cgroups for isolation and resource limits
- Image layers and copy-on-write filesystem
- Dockerfile best practices (layer ordering, exec-form ENTRYPOINT)
- Docker volumes for persistent storage
- Multi-stage builds, non-root users, and health checks
- Container registries for image distribution

Setup Check

Checking Docker Desktop installation — encountered a virtualization support error on Windows. This prompted setting up Docker via WSL2 instead.

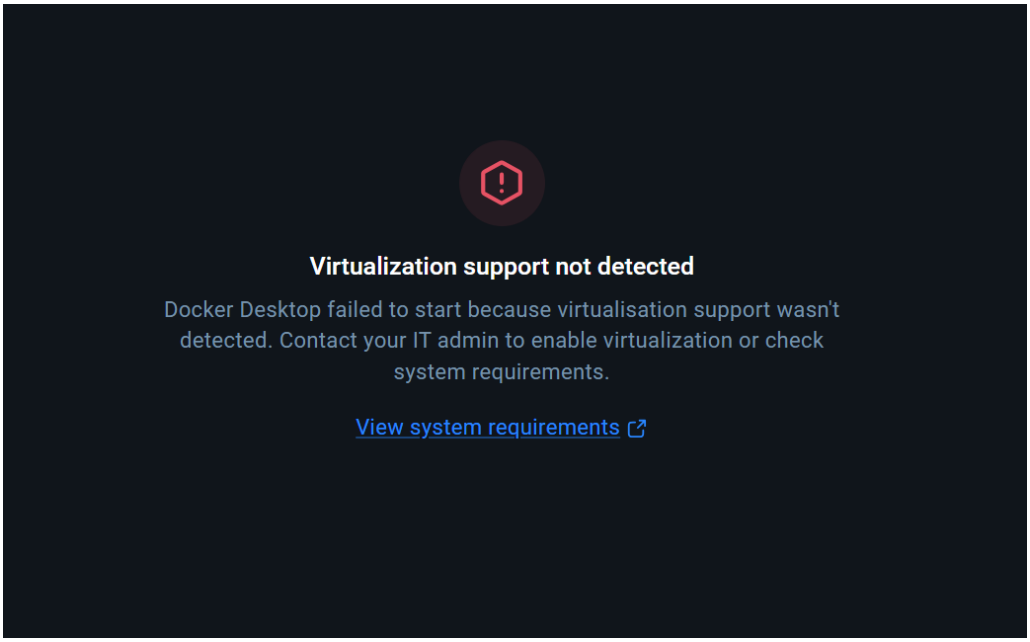


Figure 1: Docker Desktop — Virtualization support not detected

Step 1: Run Your First Container

Docker Installation Check in WSL2

Attempting to run Docker commands in WSL2. Initially, Docker was not found in the WSL2 distro, requiring manual installation of Docker Engine inside Ubuntu.

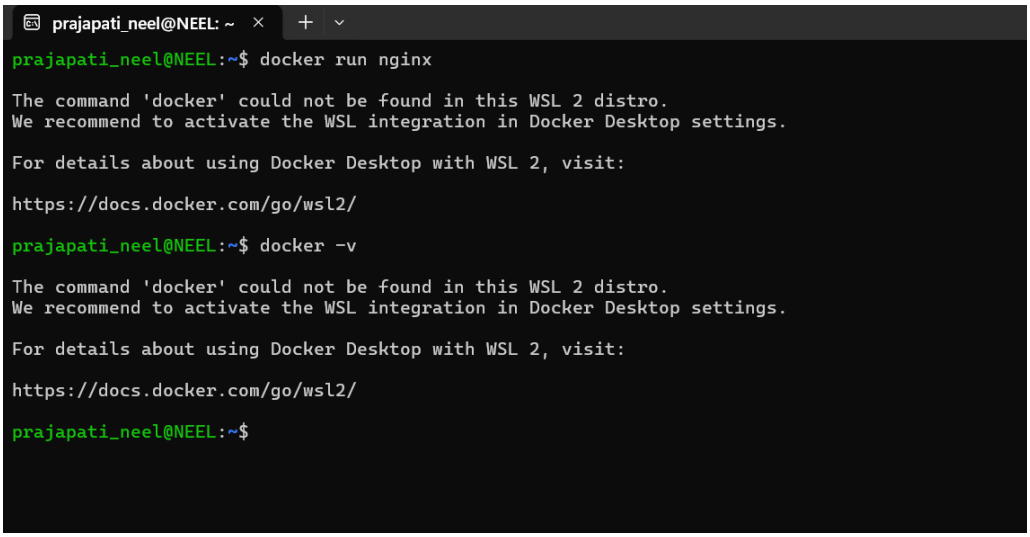


Figure 2: Docker not found in WSL2 — installing Docker Engine

Running First Nginx Container

After installing Docker Engine in WSL2, successfully ran the first Nginx container using `docker run -d -name web -p 8080:80 nginx:alpine` and verified it with `curl`.

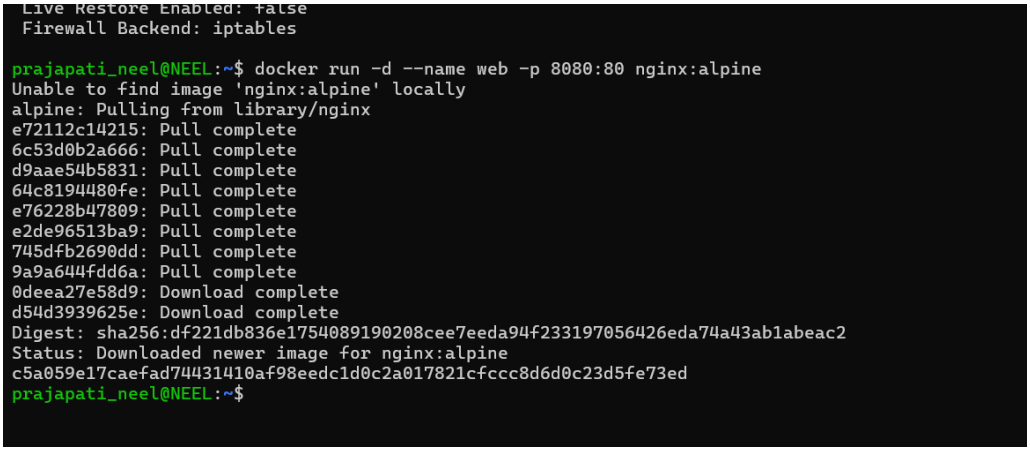


Figure 3: Running first nginx:alpine container and pulling image layers



Figure 4: Verifying nginx container with curl — "Welcome to nginx!"

Verifying with docker ps, docker logs

Verified the running Nginx container using `docker ps`, `docker ps -a`, and `docker logs web`. The container is up and serving the default Nginx welcome page.

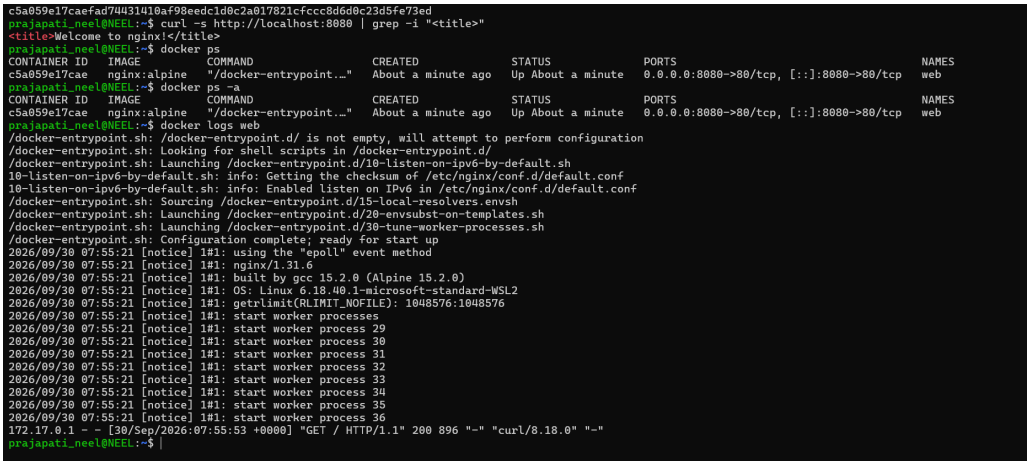


Figure 5: docker ps, docker ps -a, and docker logs output

Step 2: Images vs Containers

Pull, Create, Start Stages

Demonstrated the three stages of `docker run`: pull (downloading the image), create (preparing the container), and start (running it). Showed that `docker run = pull + create + start`.

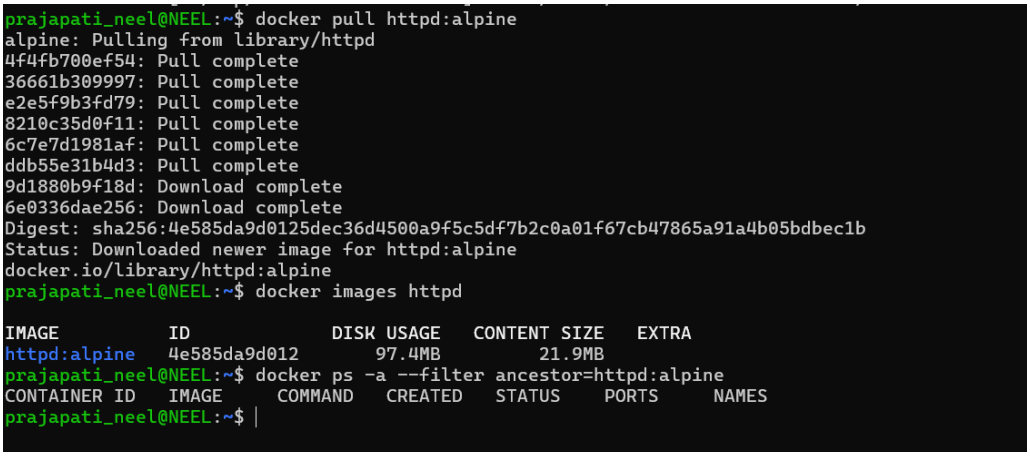


Figure 6: Pulling httpd:alpine and creating container separately

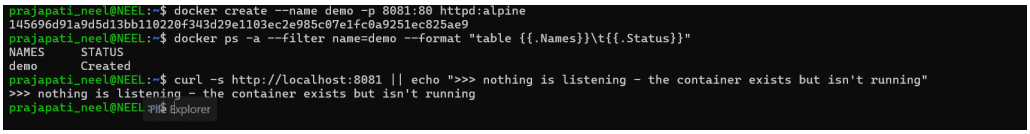


Figure 7: Starting the container and verifying with curl

One Image, Many Containers

Explored creating multiple containers from a single image, container naming conflicts, and the difference between `docker stop` (polite, 10s timeout) vs `docker kill` (instant).

```
prajapati_neel@NEEL:~$ docker start demo
demo
prajapati_neel@NEEL:~$ curl -s http://localhost:8081 | head -n 1
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
prajapati_neel@NEEL:~$
```

Figure 8: Multiple containers from one image and stop vs kill timing

Step 3: Look Inside a Container

Exploring Container Internals

Used `docker exec -it web sh` to get a shell inside the container. Observed that the container sees itself as PID 1 with its own hostname and filesystem (Alpine Linux), while sharing the same kernel as the host.

```
prajapati_neel@NEEL:~$ curl -s http://localhost:8081 | head -n 1
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
prajapati_neel@NEEL:~$ docker run -d --name demo httpd:alpine
docker: Error response from daemon: Conflict. The container name "/demo" is already in use by container "145696d91a9d5d13bb110220f343d29e1103ec2e985c07e1
a9251ec825ae9". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
prajapati_neel@NEEL:~$ docker run -d httpd:alpine
96c79f230eb0fc4d1a9f1fac9ec88c58245514 3d260212e193e4bd580f887ab3
prajapati_neel@NEEL:~$ docker run -d httpd:alpine
045dccc734eba7e45f2261e30fd084ef301b98ef9d7076edd150afa84b96601
prajapati_neel@NEEL:~$ docker ps --filter ancestor=httpd:alpine --format "{{.ID}}  {{.Names}}"
045dccc734eb  romantic_ishizaka
96c79f230eb0  wizardly_chaplygin
145696d91a9d  demo
prajapati_neel@NEEL:~$
```

Figure 9: Inside the container — hostname, ps, filesystem exploration

```
prajapati_neel@NEEL:~$ docker ps --filter ancestor=httpd:alpine --format "{{.ID}}  {{.Names}}"
045dccc734eb  romantic_ishizaka
96c79f230eb0  wizardly_chaplygin
145696d91a9d  demo
prajapati_neel@NEEL:~$ docker run -d --name sleeper1 alpine:3.20 sleep 1000
Unable to find image 'alpine:3.20' locally
3.20: Pulling from library/alpine
25f1d6b1951a: Pull complete
d4050d56ebf2: Download complete
3a030ca3f633: Download complete
Digest: sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d44e6e2a4b6bc
Status: Downloaded newer image for alpine:3.20
7a9ce6ae7f0ddc26a1ed57f570771c7b763151793e5e8f1a8ed4bcc3f05194cd
prajapati_neel@NEEL:~$ docker run -d --name sleeper2 alpine:3.20 sleep 1000
f5fb652f7855ca6c33c3ddcfdb52637eeba9b0d64ebadd81d5219d133827eab1
prajapati_neel@NEEL:~$ time docker stop sleeper1
sleeper1

real    0m3.204s
user    0m0.012s
sys     0m0.006s
prajapati_neel@NEEL:~$ time docker kill sleeper2      # rude: pulls the plug
sleeper2

real    0m0.198s
user    0m0.010s
sys     0m0.006s
```

Figure 10: Proving same kernel — container uses host's kernel (not a VM)

Step 4: Who Actually Does the Work?

Docker Architecture

Explored the Docker architecture by examining `dockerd`, `containerd`, and the shim process. Demonstrated starting a container using `curl` to talk directly to the Docker daemon's REST API via the Unix socket.

```
/ # exit
prajapati_neel@NEEL:~$ mkdir -p ~/docker-lab/campusconnect && cd ~/docker-lab/campusconnect
prajapati_neel@NEEL:~/docker-lab/campusconnect$ echo "flask==3.0.3" > requirements.txt
prajapati_neel@NEEL:~/docker-lab/campusconnect$
prajapati_neel@NEEL:~/docker-lab/campusconnect$ cat > app.py <<'EOF'
import argparse, os, signal, socket, sys
from flask import Flask, jsonify

app = Flask(__name__)
DATA_DIR = os.environ.get("DATA_DIR", "/data")
COUNT_FILE = os.path.join(DATA_DIR, "visits.txt")
APP_VERSION = os.environ.get("APP_VERSION", "v1")

def read_count():
    try:
        with open(COUNT_FILE) as f:
            return int(f.read().strip() or 0)
    except FileNotFoundError:
        return 0

def write_count(n):
    os.makedirs(DATA_DIR, exist_ok=True)
    with open(COUNT_FILE, "w") as f:
        f.write(str(n))

@app.route("/")
def index():
    n = read_count() + 1
    write_count(n)
    return jsonify(message="Hello from CampusConnect!", visits=n,
                   hostname=socket.gethostname(), pid=os.getpid(),
                   version=APP_VERSION)

@app.route("/health")
def health():
    return jsonify(status="ok")

def handle_sigterm(signum, frame):
    print("Received SIGTERM - shutting down gracefully", flush=True)
    sys.exit(0)

signal.signal(signal.SIGTERM, handle_sigterm)

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--port", type=int, default=8000)
    args = parser.parse_args()
    print(f"Starting CampusConnect {APP_VERSION} on port {args.port}", flush=True)
    app.run(host="0.0.0.0", port=args.port)
EOF
prajapati_neel@NEEL:~/docker-lab/campusconnect$
```

Figure 11: Creating a container via the Docker API with curl

Stopping the Daemon

Stopped the Docker daemon with `systemctl stop docker` and observed that the CLI could no longer communicate. Restarted the daemon and cleaned up.

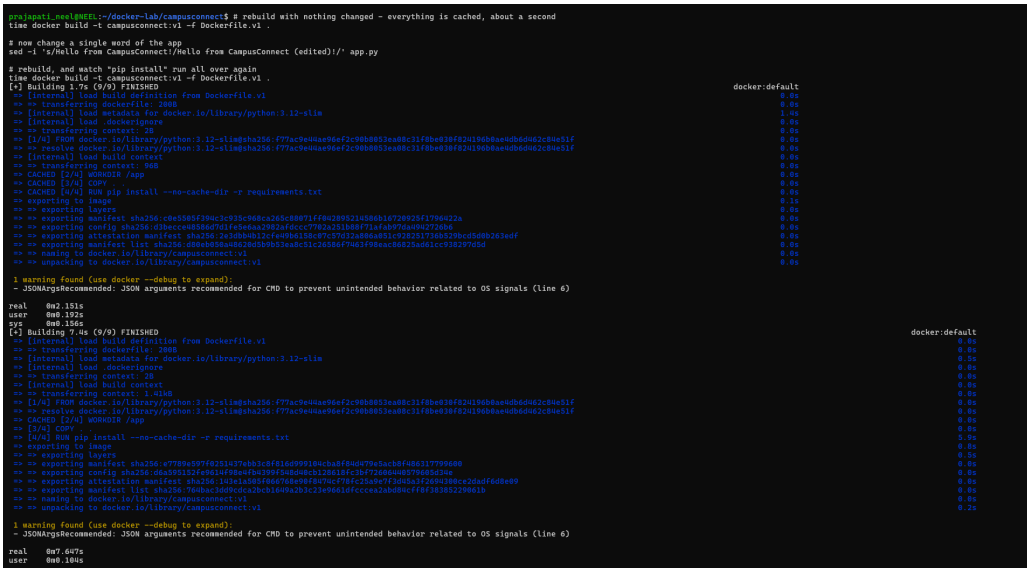


Figure 12: Daemon stopped — CLI becomes useless, then restarted

Step 5: Setting Limits (cgroups)

CPU and memory limits using cgroups. Created containers with resource limits (`-cpus=0.5`, `-memory=128m`). Verified that the kernel enforces these limits by reading cgroup files. Observed a CPU-burning container capped at 50%.



Figure 13: Setting CPU and memory limits, reading cgroup files

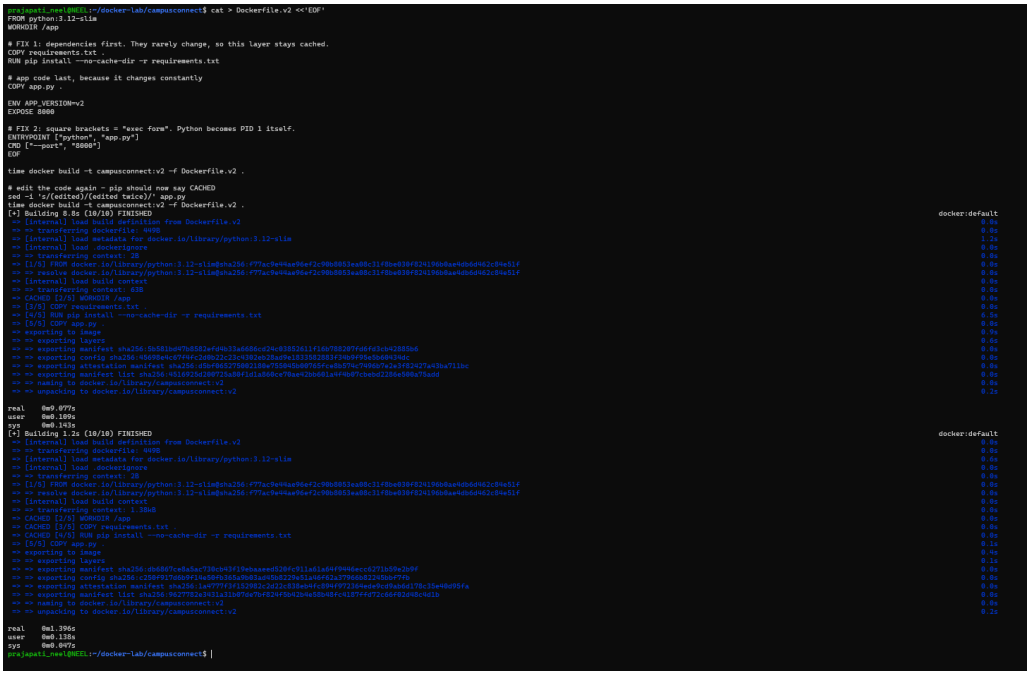


Figure 14: CPU burner container capped at 50%, OOM kill test

Step 6: Layers and Copy-on-Write

Demonstrated copy-on-write by modifying a file in one container (cow1) and showing that the twin container (cow2) remains unaffected. Verified the thin writable layer cost using `docker ps -s`.



Figure 15: Copy-on-write: modifying cow1, cow2 stays unchanged

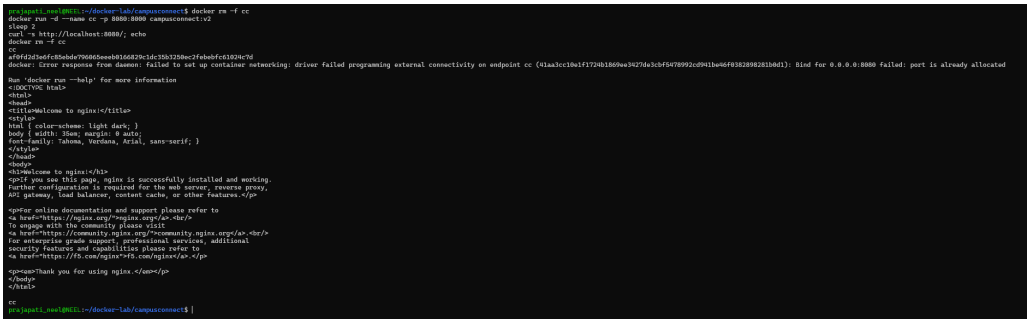


Figure 16: docker diff and docker ps -s showing layer costs

Step 7: Build Your Own Image

Writing a Dockerfile for CampusConnect

Created the CampusConnect visit counter app, wrote `Dockerfile.v1`, built the image, and ran it. Identified two bugs: slow rebuilds due to poor layer ordering, and shell-form CMD preventing graceful shutdown.

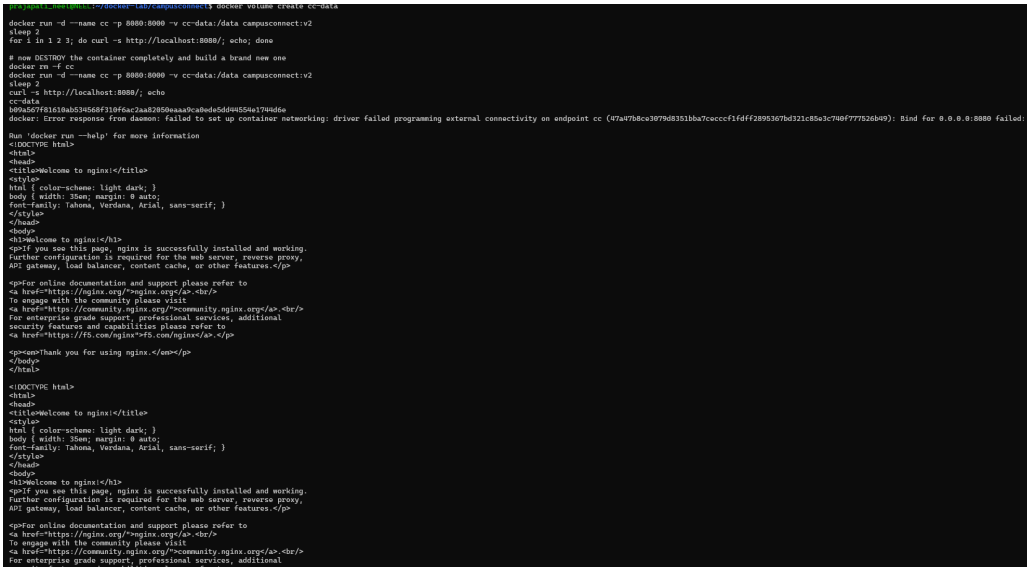


Figure 17: Building CampusConnect v1 and identifying the shell-form bug

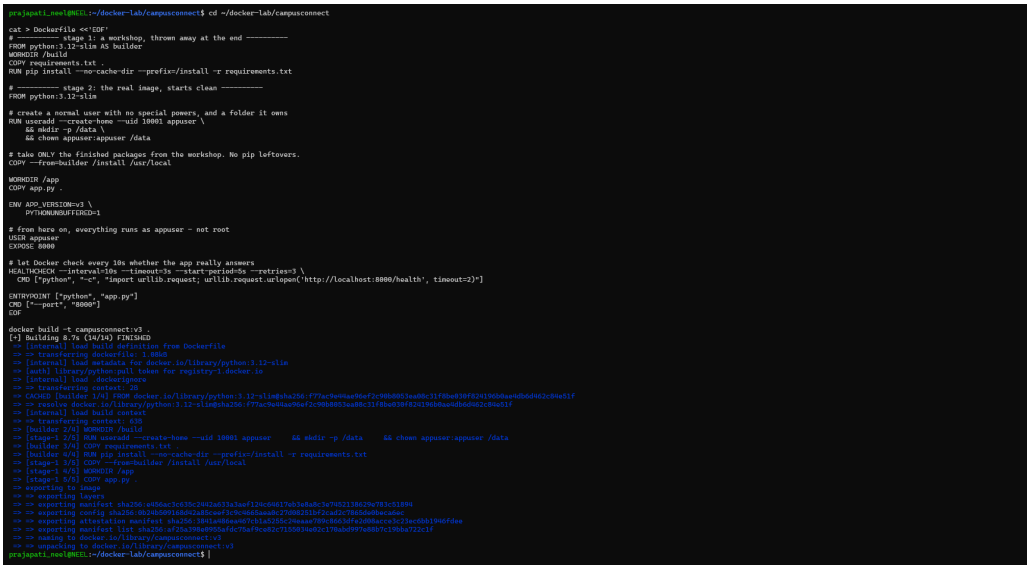


Figure 18: 10-second stop time and exit code 137 — PID 1 problem

Fixing the Dockerfile

Created `Dockerfile.v2` with fixes: dependencies copied before app code (for cache efficiency), and exec-form ENTRYPOINT so Python becomes PID 1 and receives SIGTERM properly. Verified graceful shutdown.



Figure 19: v2 with exec-form: clean shutdown in 0.2s, exit code 0

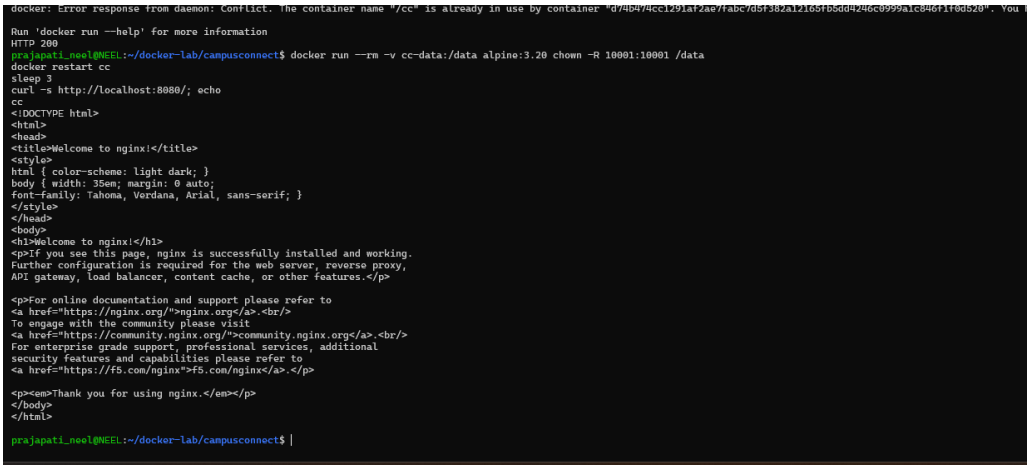


Figure 20: SIGTERM received gracefully — "Received SIGTERM" in logs

Step 8: Keep Your Data (Volumes)

Demonstrated data loss when containers are deleted. Fixed it using Docker volumes (`docker volume create cc-data`) and bind mounts. Showed that the visit counter

persists across container recreations.

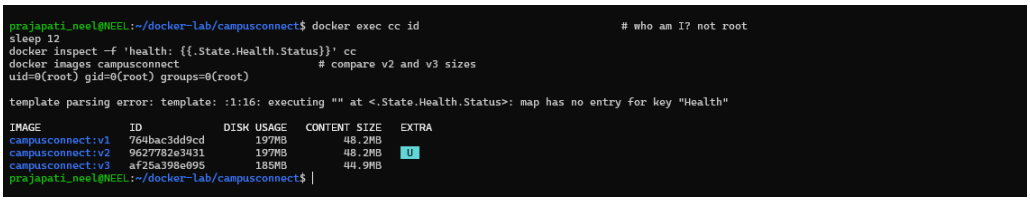


Figure 21: Volume mount — data persists across container deletion



Figure 22: Bind mounts and volume inspection

Step 9: Ship It Like a Pro

Built a production-ready Dockerfile with multi-stage build, non-root user (`appuser`), and HEALTHCHECK. Published the image to a local Docker registry and pulled it back for deployment.

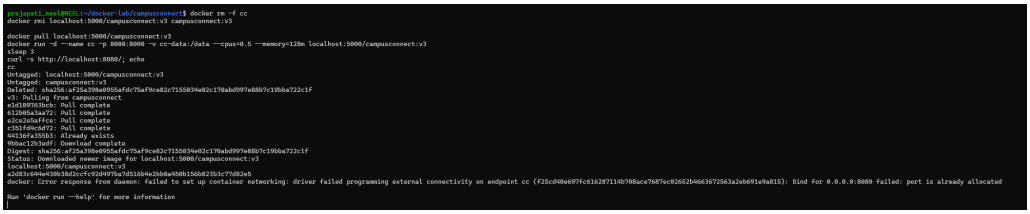


Figure 23: Multi-stage build, non-root user, and health check

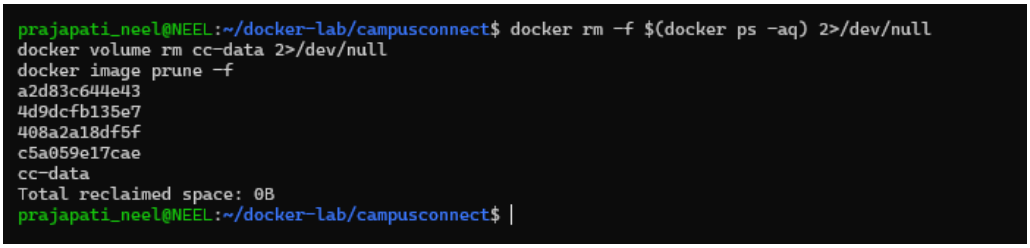


Figure 24: Pushing to local registry and pulling back for deployment

Conclusion

This hands-on lab provided comprehensive practical experience with Docker containerization. Through the nine steps, the following key learnings were achieved:

1. A container is a single process wrapped with kernel namespaces and cgroups — not a virtual machine. It shares the host's kernel and starts in a fraction of a second.
2. `docker run` is a convenience shortcut for `pull + create + start`. Understanding this decomposition is essential for advanced usage and API-based automation.
3. Inside a container, the main process is PID 1. Outside, it appears as a regular process in the host's process list — same kernel, different view (namespaces).
4. The Docker CLI is just a client that sends HTTP requests to the Docker daemon (`dockerd`) over a Unix socket. The daemon delegates to `containerd`, which uses `runc` to create containers.
5. Cgroups provide resource limits (CPU, memory). Exit code 137 indicates a force-kill, often due to OOM or ignored SIGTERM.
6. Images consist of shared read-only layers. Each container adds a thin writable layer (copy-on-write), making containers extremely lightweight.
7. Dockerfile best practices: place rarely-changing instructions first (dependencies before code), and always use exec-form (square brackets) for ENTRYPOINT/CMD so the application receives signals properly.
8. Volumes provide persistent storage that survives container deletion. Bind mounts allow direct access to host filesystem directories.
9. Production images should use multi-stage builds, run as non-root users, include health checks, and be published to a registry for deployment.

The lab successfully demonstrated the complete Docker workflow: from running a first container to building, optimizing, and publishing a production-ready image.